

Министерство науки и высшего образования Российской Федерации
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ
ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
«Национальный исследовательский университет ИТМО»
(Университет ИТМО)

Факультет программной инженерии и компьютерной техники

Образовательная программа: Системное и прикладное программное
обеспечение

Направление подготовки (специальность): 09.04.04 Программная
инженерия

Наименование практики: Научно-исследовательская работа

О Т Ч Е Т

о научно-исследовательской работе

Тема задания:

**Сравнительный анализ архитектурных решений
серверной платформы текстовой MMORPG**

Обучающийся Горинов Даниил Андреевич, № группы Р4116

Руководитель практики от университета: Белозубов Александр
Владимирович,
руководитель в ИТМО

Дата 21.06.2026

Санкт-Петербург
2026

СОДЕРЖАНИЕ

СПИСОК СОКРАЩЕНИЙ И УСЛОВНЫХ ОБОЗНАЧЕНИЙ	4
1 Введение	4
2 План-график выполнения работы	6
3 Выполнение этапа 2: анализ предметной области и постановка задачи	6
3.1. Предметная область	6
3.2. Объект, предмет, цель и задачи исследования	8
3.3. Критерии анализа и источниковая база	9
4 Выполнение этапа 3: исследование архитектурных решений и сравнительный анализ	10
4.1. Предварительные архитектурные гипотезы	10
4.2. Экспериментальный стенд	11
4.3. Результаты по производительности и масштабируемости . . .	13
4.4. Экспериментальная сравнительная матрица	15
4.5. Обоснование архитектурной комбинации	17
5 Выполнение этапа 4: верификация выводов и формирование рекомендаций	18
5.1. Матрица отказов и проверка надёжности	18
5.2. Evidence trace	19
5.3. Наблюдаемость и проверяемые метрики	20
5.4. Проверка терминологии, источников и ограничений валидности	21
5.5. Практические рекомендации	22
6 Выполнение этапа 5: оформление отчётных документов	22
7 Заключение	23
8 Список использованных источников	25

ПРИЛОЖЕНИЕ 1 — Репозиторий, команды запуска и структура стенда	30
ПРИЛОЖЕНИЕ 2 — Raw-сводка экспериментальных артефактов	31
ПРИЛОЖЕНИЕ 3 — Протокол проверки источников, сборки и оформления	32

СПИСОК СОКРАЩЕНИЙ И УСЛОВНЫХ ОБОЗНАЧЕНИЙ

Сокращения и условные обозначения

Сокращение	Значение
MMORPG	Massively Multiplayer Online Role-Playing Game; в отчёте рассматривается текстовая многопользовательская ролевая онлайн-игра с серверной обработкой команд, чата и игровых транзакций.
API	Программный интерфейс взаимодействия клиента, внешней платформы или worker-процесса с серверной платформой.
WebSocket	Двусторонний протокол realtime-взаимодействия, определённый RFC 6455 [1].
Outbox	Таблица исходящих доменных событий, записываемая в одной транзакции с изменением бизнес-состояния.
Inbox	Механизм фиксации идентификаторов входящих сообщений для защиты от повторной обработки callback и команд.
Saga	Последовательность локальных транзакций с явными состояниями, повторными попытками и компенсациями.
Pub/Sub	Модель публикации и подписки; в Redis Pub/Sub используется для короткоживущих live-уведомлений.
Streams	Redis Streams; журнал событий с возможностью чтения истории и восстановления после разрыва соединения.
Source of truth	Основной источник достоверного состояния; в работе эту роль для экономики и прогресса выполняет PostgreSQL.
DLQ	Dead-letter queue, очередь сообщений, которые не удалось обработать после допустимого числа попыток.
p50, p95, p99	50-й, 95-й и 99-й процентиля задержки обработки команд в экспериментальном стенде.

1 Введение

В рамках научно-исследовательской работы я продолжил исследование архитектурных паттернов серверных платформ для текстовых многопользовательских онлайн-игр, начатое в аналитическом обзоре прошлого семестра [2]. В обзорной работе я систематизировал основные паттерны, источники и ограничения предметной области. В данном отчёте я перешёл от обзорного сопоставления к практической проверке: разработал исследовательский стенд, выполнил серию нагрузочных и fault-injection экспериментов и на этой основе уточнил архитектурные рекомендации.

Предметная область включает игровые системы, в которых основная

нагрузка возникает не от графического рендеринга, а от обработки пользовательских команд, чата, realtime-уведомлений, внутриигровой экономики, повторных callback внешних платформ и фоновых процессов. Для таких систем существенны высокая нагрузка, отказоустойчивость, наблюдаемость, контролируемая стоимость эксплуатации и корректность бизнес-состояния.

Цель работы — обоснованно определить архитектурную комбинацию для серверной платформы текстовой MMORPG, которая сохраняет корректность игрового состояния, поддерживает realtime-взаимодействие и допускает рост нагрузки без преждевременного перехода к избыточно распределённой архитектуре.

Для достижения цели я решил следующие задачи:

1. проанализировал предметную область серверных платформ текстовых многопользовательских онлайн-игр в социальных сетях, мессенджерах и web-клиентах;
2. определил объект, предмет, цель, задачи и границы исследования;
3. сформировал критерии сравнения: масштабируемость, надёжность, производительность, сложность внедрения, стоимость эксплуатации и наблюдаемость;
4. подобрал и систематизировал научные и промышленные источники;
5. исследовал event-driven architecture, Transactional Outbox/Inbox, Saga, WebSocket-инфраструктуру, брокеры сообщений, кэширование, PostgreSQL и Redis;
6. разработал самостоятельный экспериментальный стенд, опубликованный в репозитории [3];
7. выполнил серии нагрузочных и fault-injection экспериментов;
8. сформировал итоговую экспериментальную матрицу, evidence trace и практические рекомендации.

Методику работы я строил как трассируемую цепочку: критерий сравнения — сценарий стенда — метрика — результат — вывод — рекомендация. Такой порядок позволил не ограничиваться качественными оценками паттернов, а проверять каждый сильный вывод через источник, реализованный сценарий стенда или экспериментальный артефакт. Оформление отчёта я выполнил с учётом требований учебно-методического пособия Университета

ИТМО по организации и проведению производственной практики магистрантов: титульный лист, содержание, список сокращений, введение, основная часть, заключение, список источников и приложения; формат A4, Times New Roman 14 пт, полуторный интервал, поля 30/15/20/20 мм [4].

2 План-график выполнения работы

Работу я выполнял по плану-графику, приведённому в таблице 2.1. Этапы 2–4 образовали содержательную основу исследования. На этапе 5 я оформлял письменный отчёт, переносил ключевые результаты в основной текст и проверял воспроизводимость материалов. Этап 6 относится к пост-отчётному оформлению отзыва руководителя в модуле «Практика».

Таблица 2.1 – План-график выполнения НИР

Этап	Срок	Содержание	Результат	Статус
1	02.03.2026	Инструктаж обучающегося по требованиям охраны труда, техники безопасности, пожарной безопасности и внутреннего трудового распорядка	Факт прохождения организационного инструктажа	Выполнено
2	03.03.2026– 23.03.2026	Анализ предметной области и постановка задачи исследования	Объект, предмет, цель, задачи, критерии анализа и обзор источников	Выполнено
3	24.03.2026– 26.05.2026	Исследование архитектурных решений и сравнительный анализ подходов	Сценарии стенда, экспериментальная матрица и обоснование архитектурной комбинации	Выполнено
4	27.05.2026– 21.06.2026	Верификация выводов и формирование рекомендаций	Проверенные источники, терминология, evidence trace, fault-сценарии и рекомендации	Выполнено
5	02.03.2026– 21.06.2026	Оформление отчётных документов	Письменный отчёт, приложения со вспомогательными материалами, сборка PDF	Выполнено
6	22.06.2026– 27.06.2026	Получение отзыва руководителя практики	Отзыв и оценка руководителя в модуле «Практика»	Пост-отчётный этап

3 Выполнение этапа 2: анализ предметной области и постановка задачи

3.1 Предметная область

На первом содержательном этапе я вернулся к аналитическому обзору, подготовленному в предыдущем семестре [2]. В нём уже были выделены основные классы архитектурных решений для текстовых многопользовательских онлайн-игр: Outbox/Inbox, Saga, WebSocket-доставка, кэширование,

PostgreSQL и Redis. В текущей НИР мне нужно было уточнить не только перечень паттернов, но и то, какие свойства этих паттернов действительно проверяются в условиях, близких к серверной платформе текстовой MMORPG.

Текстовую MMORPG я рассматривал как многопользовательскую систему, в которой пользователь взаимодействует с игровым миром через команды, сообщения, уведомления, кнопки интерфейса социальной сети или мессенджера. В отличие от графических онлайн-игр, такая система не требует постоянной синхронизации трёхмерной сцены, но предъявляет высокие требования к серверной логике: экономические операции должны быть транзакционными, чат должен доставляться с малой задержкой, состояние игрока должно восстанавливаться после сбоя, а повторный callback внешней платформы не должен приводить к повторному начислению валюты или предмета.

После повторного анализа источников я разделил сценарии платформы по критичности состояния и требованиям к доставке. Исследования backend-архитектур MMOG показывают, что серверная часть многопользовательской игры должна совмещать persistence, сетевую доставку, масштабирование и контроль согласованности состояния [5, 6]. Работы по transaction models и consistency requirements для MMOG подчёркивают, что действия игроков конкурируют за игровые ресурсы, предметы, валюту и состояние мира, поэтому простая обработка сообщений без модели согласованности недостаточна [7—10]. С учётом этого я выделил классы сценариев серверной платформы, приведённые в таблице 3.1.

Таблица 3.1 – Классы сценариев серверной платформы текстовой MMORPG

Класс сценариев	Типичные операции	Архитектурное требование
Игровые команды	перемещение по локациям, выполнение задания, получение награды	транзакционная фиксация состояния, идемпотентность повторной команды, аудит результата
Чат и сообщения	сообщения комнаты, личные сообщения, системные уведомления	realtime-доставка, fan-out активным клиентам, отделение transient-сообщений от критичного состояния
Экономика и инвентарь	списание валюты, покупка предмета, выдача награды, обмен	строгие инварианты, уникальные ключи, локальные транзакции, Saga для цепочек с компенсациями
Интеграции и callback	ответы внешней платформы, повторные webhook, отложенные задания	Inbox, correlation id, устойчивость к повторной доставке
Realtime-состояние	online presence, обновления интерфейса, reconnect клиента	WebSocket, heartbeat, backpressure, replay или snapshot после разрыва соединения
Фоновые процессы	начисление периодических наград, события мира, обработка очередей	Outbox, worker-пулы, метрики lag/retry/DLQ, безопасная остановка и повторный запуск

3.2 Объект, предмет, цель и задачи исследования

Объектом исследования я определил серверные платформы текстовых MMORPG, рассчитанные на взаимодействие пользователей через социальные сети, мессенджеры и web-клиенты. Предметом исследования я выбрал архитектурные паттерны и инфраструктурные решения, обеспечивающие обработку игровых событий, чата, realtime-взаимодействия и внутриигровых транзакций.

Цель исследования я сформулировал как выбор архитектурной комбинации, которая обеспечивает корректность критичного состояния, поддерживает realtime-доставку и остаётся реализуемой при росте нагрузки. Для достижения цели было недостаточно перечислить решения по литературе. Поэтому я сразу зафиксировал проверяемые сценарии, характерные для текстовой MMORPG: повторная доставка callback, сбой после публикации события, потеря ephemeral-сообщения, reconnect клиента, устаревший кэш и частичная экономическая операция.

3.3 Критерии анализа и источниковая база

Критерии сравнения я взял из задания на практику и уточнил для экспериментальной проверки. В таблице 3.2 каждому критерию сопоставлены проверяемые сценарии и метрики стенда. Такой способ был нужен для того, чтобы итоговые рекомендации были связаны с наблюдаемыми данными, а не только с общими характеристиками технологий.

Таблица 3.2 – Критерии сравнения и способ их проверки

Критерий	Содержание критерия	Сценарий стенда	Метрики
Масштабируемость	способность увеличивать число команд, сообщений, worker-процессов и подключений	серии 100/500/1000 для sync, outbox и chat	throughput, error rate, stream length
Надёжность	устойчивость к повторам, падению relay, потере transient-сообщений, reconnect и stale cache	duplicate callback, publish-no-ack, Pub/Sub loss, WebSocket replay, Saga, cache stale	duplicates, outbox pending, replay received, stale, completed/compensated
Производительность	влияние решения на задержку и пропускную способность	latency-серии sync/outbox/chat	p50, p95, p99, throughput
Сложность внедрения	число компонентов, состояний, контрактов и worker-путей	реализованные компоненты стенда	component/state/contract count, Saga states
Стоимость эксплуатации	инфраструктурная и операционная цена решения	stateful-зависимости, retained history, backlog	Redis, PostgreSQL, worker, stream length, attempts
Наблюдаемость	возможность доказать или опровергнуть архитектурный вывод метрикой	/metrics/summary, CSV/JSON	backlog age, lag, duplicates, Saga duration, cache stale

Источники я использовал по принципу применимости к конкретному утверждению. Рецензируемые работы применялись для анализа MMOG persistence, consistency и transaction models [5—10]. Книги и паттерны распределённых систем я использовал для терминов и интеграционных решений [11—15]. RFC и официальные документации применялись для свойств конкретных технологий [1, 16—19]. Индустриальные материалы Microsoft, Microservices.io, Event-Driven.io, InfoQ, Ably и WebSocket.org использовались для современных эксплуатационных практик [20—26]. Русскоязычные публикации по WebSocket/Socket.IO я применял как дополнительную сверку терминологии realtime-приложений [27, 28].

4 Выполнение этапа 3: исследование архитектурных решений и сравнительный анализ

4.1 Предварительные архитектурные гипотезы

После постановки задачи я разложил рассматриваемые решения по ролям, чтобы не сравнивать их как взаимоисключающие технологии. PostgreSQL я рассматривал как источник достоверного состояния для экономики, прогресса, инвентаря и аудита; Transactional Outbox — как способ атомарно связать изменение состояния и исходящее событие; Inbox — как защиту от повторной доставки; Redis Streams — как persisted event log для replay; Redis Pub/Sub — как transient-канал live fan-out; WebSocket gateway — как realtime-интерфейс клиента; Saga — как механизм координации долгих экономических цепочек; cache-aside — как ускорение чтения некритичных данных.

Эти роли согласуются с источниками: Outbox устраняет проблему двойной записи, но допускает повторную публикацию при сбое relay между publish и отметкой доставки [21, 22]; Pub/Sub не хранит историю для позднего подписчика [17]; Redis Streams предоставляет журнал для чтения истории [16]; WebSocket является транспортом realtime-обмена, но не заменяет источник истины или журнал событий [1, 25]; Saga применяется для последовательности локальных транзакций с компенсациями [23, 24].

Предварительная гипотеза состояла в том, что серверная платформа текстовой MMORPG должна строиться как комбинация перечисленных решений, а не как выбор одного универсального брокера, одной базы данных или одного realtime-протокола. Эту гипотезу я затем проверял на стенде. Архитектурная схема проверяемой комбинации приведена на рисунке 4.1.

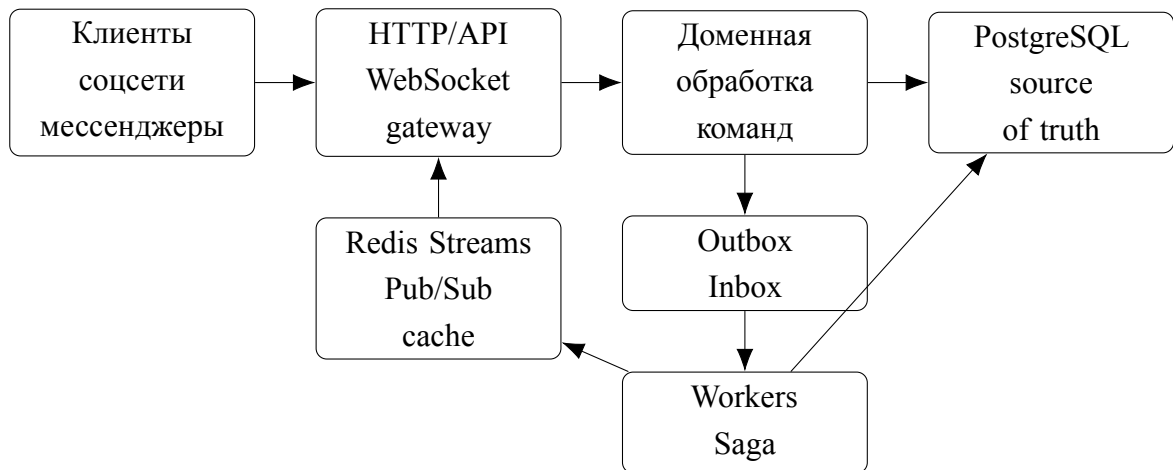


Рисунок 4.1 – Архитектурная комбинация, проверенная на исследовательском стенде

4.2 Экспериментальный стенд

Для проверки гипотезы я разработал самостоятельный стенд, опубликованный в репозитории [3]. Для backend-части я использовал Go, потому что стенд должен был быстро поднимать HTTP API, worker-процессы и WebSocket endpoint без лишней инфраструктуры. Для сценариев нагрузки, fault-injection и построения графиков я использовал Python, поскольку в нём удобно собирать CSV/JSON-артефакты и строить повторяемые визуализации. PostgreSQL и Redis я запускал вместе с backend и worker через Docker Compose, чтобы проверка не зависела от внешних сервисов.

Стенд моделирует минимальную серверную платформу текстовой игры: команды игрока изменяют баланс золота; Inbox фиксирует идентификаторы входящих сообщений; Outbox сохраняет исходящие события; worker публикует события в Redis Streams и Pub/Sub; WebSocket-клиент проверяет replay; Saga-worker завершает или компенсирует покупку; cache-aside endpoint демонстрирует устаревшее чтение.

Первой технической сложностью стал выбор границы между локальной транзакцией PostgreSQL и асинхронной публикацией события. Если публиковать событие напрямую из обработчика команды, появляется риск двойной записи: состояние уже изменено, а событие не доставлено, либо наоборот. Поэтому я вынес публикацию в Outbox и отдельно проверил сценарий `publish_no_ack`, где relay успевает опубликовать событие, но не успевает отметить его доставленным. Этот сценарий был важен для отчёта,

потому что он показывает не абстрактный риск, а конкретное место, где без идемпотентного потребителя возникает дубль.

Вторую сложность вызвал выбор модели realtime-доставки. Redis Pub/Sub удобен для live fan-out, но не хранит историю для клиента, который подключился позднее. Redis Streams, наоборот, сохраняет события и позволяет выполнить replay, но требует контроля длины stream и политики хранения. Поэтому я реализовал оба механизма: Pub/Sub использовал как быстрый transient-канал, а Streams — как журнал для восстановления после reconnect. Для WebSocket-проверки я добавил отдельный клиент, который подключается с начальным `last=0-0` и подтверждает, что события можно получить повторно из stream.

Отдельно я реализовал Saga-сценарий и cache-aside сценарий. Saga понадобилась для проверки долгой экономической операции, где возможны два корректных исхода: завершение и компенсация. Cache-aside был включён в стенд не как рекомендация для экономики, а как контролируемая демонстрация риска stale read: кэш может вернуть старое значение, если источник истины уже изменился.

Стенд не претендует на моделирование всего промышленного кластера MMORPG. Его назначение — воспроизвести причинные свойства архитектурных паттернов в контролируемой среде. Поэтому абсолютные значения latency и throughput трактуются как локальные экспериментальные значения, а не как универсальный benchmark. Достоверность выводов обеспечивается повторяемостью сценариев, фиксированным seed 4116, проверкой инвариантов и сохранением CSV/JSON-артефактов.

Таблица 4.1 – Сценарии экспериментального стенда

Сценарий	Что проверяется	Метрики	Артефакты
sync transaction	базовая локальная транзакция PostgreSQL	p50/p95/p99, throughput, error rate	scenario runs, summary
outbox transaction	запись Outbox и асинхронная публикация worker	p50/p95/p99, backlog, attempts	summary
chat fan-out	запись и доставка чатовых сообщений	latency, throughput, stream length	scenario runs
duplicate callback	повторная доставка одной команды	duplicates, final gold	summary
publish-no-ack	публикация события без отметки доставки	redis duplicates, pending backlog	summary
pubsub loss	потеря сообщения для позднего подписчика	delivered flag	summary
cache stale	устаревшее cache-aside чтение	cached gold, actual gold	summary
saga	успешная и компенсированная покупка	completed, compensated, duration	summary
websocket replay	восстановление событий после reconnect	replay received	summary

4.3 Результаты по производительности и масштабируемости

После разработки прототипа я развернул тестовое окружение и 21.06.2026 выполнил нагрузочную часть эксперимента. Для сценариев `sync_transaction`, `outbox_transaction` и `chat_fanout` я использовал размеры нагрузки 100, 500 и 1000 операций; каждый размер выполнил 5 раз. Итоговые агрегированные значения приведены в таблице 4.2. Ошибки во всех строках равны нулю, что важно для интерпретации `throughput`: серия не скрывает падения запросов за счёт уменьшения числа успешных операций.

Таблица 4.2 – Агрегированные результаты нагрузочных серий

Сценарий	Нагрузка	Повт.	p50, мс	p95, мс	p99, мс	throughput, req/s	error rate
chat fan-out	100	5	0,314	0,605	0,946	762,32	0
chat fan-out	500	5	0,275	0,501	0,843	911,61	0
chat fan-out	1000	5	0,266	0,461	0,658	966,00	0
outbox transaction	100	5	0,470	0,832	0,965	741,05	0
outbox transaction	500	5	0,453	0,756	1,026	737,15	0
outbox transaction	1000	5	0,453	0,767	1,180	755,65	0
sync transaction	100	5	0,448	1,108	1,735	682,97	0
sync transaction	500	5	0,379	0,605	1,150	845,72	0
sync transaction	1000	5	0,373	0,602	0,816	858,67	0

На рисунках 4.2 и 4.3 я сопоставил тенденции `p95` и `throughput` при росте размера серии. Для локального стенда все три класса обработки оста-

лись в субмиллисекундном диапазоне p95 при нагрузке 1000 операций. Это не означает, что промышленная система будет иметь такие же задержки; вывод другой: добавление Outbox в командный путь на данном стенде не привело к неконтролируемому росту latency, а backlog после обработки был закрыт. Следовательно, Outbox допустим для критичных доменных событий при обязательном мониторинге backlog и повторных попыток.

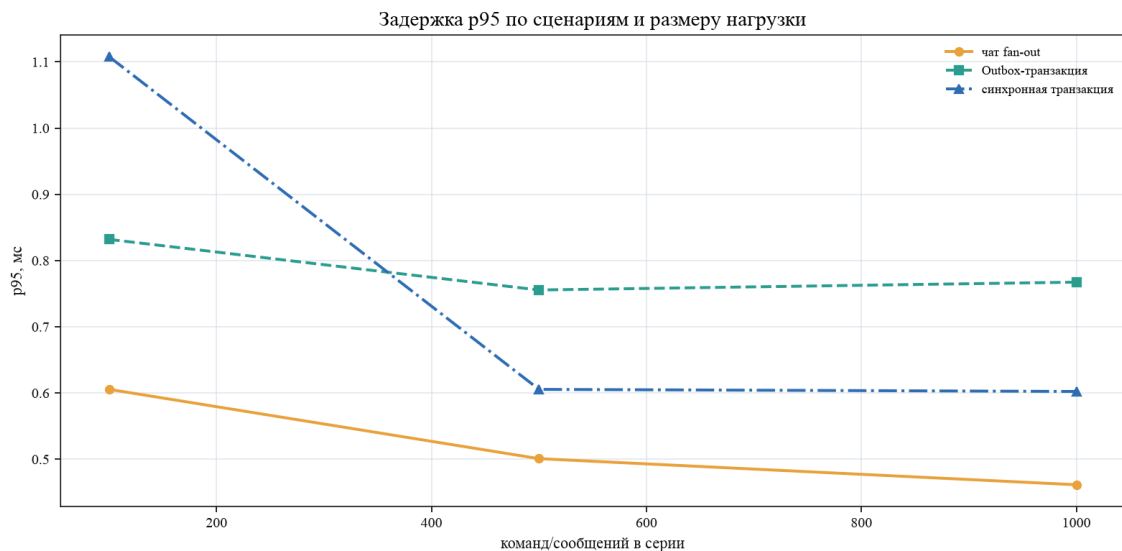


Рисунок 4.2 – Сравнение p95 latency по сценариям и размерам нагрузки

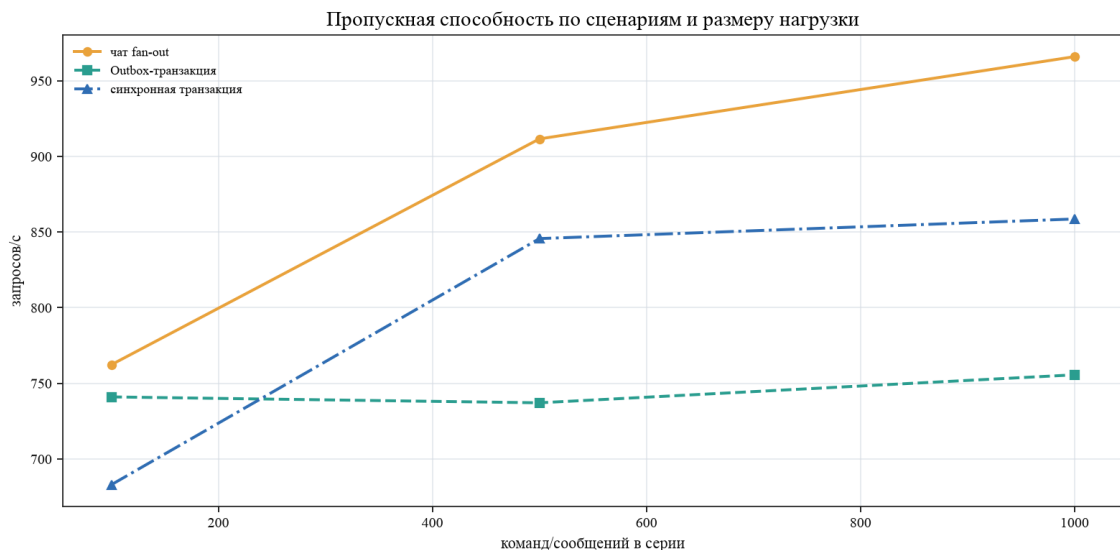


Рисунок 4.3 – Сравнение throughput по сценариям и размерам нагрузки

4.4 Экспериментальная сравнительная матрица

Итоговое сравнение я построил по результатам `comparison_matrix.csv`. Оценка от 1 до 5 отражает пригодность решения для задач текстовой MMORPG в проверенных условиях. Для производительности и масштабируемости я использовал результаты нагрузочных серий; для надёжности — fault-сценарии; для сложности внедрения, стоимости эксплуатации и наблюдаемости — реализованные компоненты стенда, число состояний, stateful-зависимости и доступные метрики.

Таблица 4.3 – Экспериментальная матрица сравнения архитектурных решений

Решение	М	Н	П	СВ	СЭ	Набл.	Итог	Экспериментальное основание	Вывод
PostgreSQL transaction	5	4	5	4	4	3	4,17	p95=0,602 мс; throughput=858,67 req/s	подходит как source of truth, но не решает fan-out событий
Transactional Outbox	5	5	5	3	3	5	4,33	p95=0,767 мс; pending=0	обеспечивает восстановимую публикацию после commit при worker и backlog-метриках
Inbox/idempotency	5	5	5	4	5	4	4,67	5 повторов дали 4 дубля и один бизнес-результат	предотвращает повторный бизнес-эффект при повторной доставке
Redis Streams	5	5	4	3	3	5	4,17	stream_len=8005; replay=3	подходит для replay и контролируемой доставки, требует политики хранения
Redis Pub/Sub	5	2	5	5	4	2	3,83	late subscriber=false	пригоден для live fan-out, но не для durable-доставки критичного состояния
WebSocket gateway	4	4	4	3	3	4	3,67	WebSocket replay=3	нужен для realtime-доставки, восстановление должно опираться на persisted stream
Saga orchestration	3	5	3	2	2	5	3,33	completed=1; compensated=1; p95=22,370 мс	подходит для долгих экономических цепочек с явными компенсациями
Cache-aside	5	2	5	4	4	3	3,83	cached=100; actual=150; stale=true	ускоряет чтение, но не может быть основанием экономических решений
Chat event fan-out	5	4	5	4	4	4	4,33	p95=0,461 мс; throughput=966,00 req/s	подходит для массовых сообщений при разделении Pub/Sub и stream history

Обозначения столбцов таблицы 4.3: М — масштабируемость, Н — надёжность, П — производительность, СВ — сложность внедрения, СЭ —

стоимость эксплуатации, Набл. — наблюдаемость. Наибольший итоговый балл получил Inbox/idempotency, что отражает критичность защиты от повторного бизнес-эффекта в системах с внешними callback. Outbox и chat fan-out имеют одинаковый высокий итоговый балл, но решают разные задачи: первый связан с восстановимой публикацией событий, второй — с массовой realtime-доставкой.

Рисунок 4.4 дополняет таблицу 4.3: он показывает не все строки матрицы, а профиль ключевых решений, определяющих итоговую архитектурную комбинацию. Такая форма позволяет увидеть инженерный компромисс: Inbox/idempotency и Outbox усиливают надёжность и наблюдаемость, Redis Pub/Sub даёт высокую скорость fan-out при слабой надёжности для поздних подписчиков, Saga имеет выраженную цену сложности, а cache-aside требует ограничения области применения из-за риска устаревшего чтения.

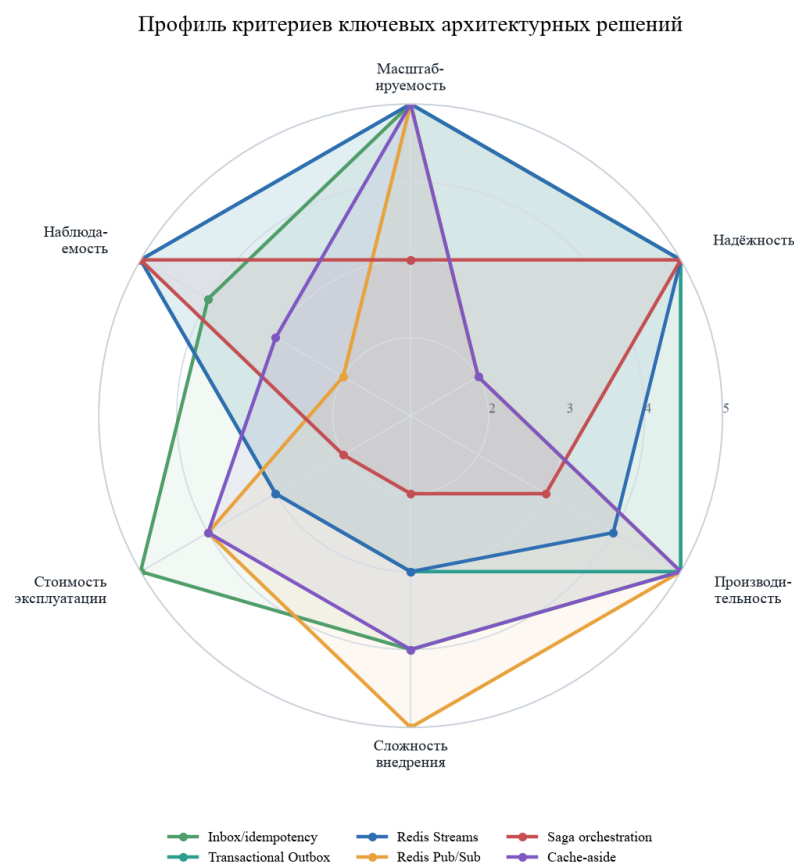


Рисунок 4.4 – Профиль критериев ключевых архитектурных решений по экспериментальной матрице

4.5 Обоснование архитектурной комбинации

На основании источников и эксперимента я обосновал следующую архитектурную комбинацию.

PostgreSQL должен оставаться источником истины для экономики, прогресса, инвентаря и аудита. Это следует из требований к локальным ACID-транзакциям, уникальным ограничениям и восстановлению состояния [18, 19]. Экспериментальная серия показала, что локальная транзакционная обработка обеспечивает устойчивую базовую задержку и нулевой error rate на стенде, но сама по себе не решает доставку событий внешним потребителям.

Transactional Outbox следует использовать для доменных событий, связанных с изменением состояния. Эксперимент `publish_no_ack` воспро-

извёл транспортный дубль: событие было опубликовано, но отметка доставки не была зафиксирована, после чего worker повторил публикацию. Это подтверждает известное ограничение Outbox: паттерн устраняет двойную запись, но не отменяет требования к идемпотентным потребителям [13, 22]. Поэтому Outbox должен применяться вместе с Inbox, уникальными ключами и дедупликацией на стороне потребителя.

Redis Pub/Sub и Redis Streams должны использоваться совместно, но для разных гарантий. Pub/Sub подходит для live fan-out активным WebSocket-шлюзам, поскольку имеет малую задержку и простую модель подписки; при этом сценарий `pubsub_loss` подтвердил, что поздний подписчик не получает уже опубликованное сообщение. Redis Streams, напротив, хранит события и обеспечивает replay: WebSocket-проверка получила 3 события после подключения с начальным `last=0-0`. Следовательно, критичное состояние восстанавливается не из Pub/Sub, а из PostgreSQL или persisted stream.

Saga-оркестрация оправдана для долгих экономических цепочек, где несколько локальных транзакций должны завершиться согласованным финалом. На стенде были получены оба наблюдаемых исхода: `completed=1` и `compensated=1`. При этом Saga имеет более высокую сложность внедрения и эксплуатации, поэтому не должна заменять простые локальные транзакции для обычных команд.

Cache-aside допустим только для данных, где устаревшее чтение не нарушает экономические инварианты. Сценарий `cache_stale` зафиксировал расхождение `cached=100` и `actual=150`; значит, кэш не может быть основанием для решений о списании валюты, выдаче предметов или правах владения.

5 Выполнение этапа 4: верификация выводов и формирование рекомендаций

5.1 Матрица отказов и проверка надёжности

После основных нагрузочных серий я перешёл к fault-injection проверкам. Я выбрал отказы, которые соответствуют типичным рискам серверной платформы текстовой MMORPG: повторная доставка callback, падение relay

после publish, потеря transient-сообщения, reconnect клиента, stale cache и частичная Saga. Результаты приведены в таблице 5.1.

Таблица 5.1 – Результаты fault-injection сценариев

Сценарий	Результат	Проверенный вывод	Архитектурное следствие
Duplicate callback	5 запросов; 4 дубля; итоговый баланс 7	повторная доставка не изменила состояние повторно	обязателен Inbox/idempotency key
Publish-no-ack	redis duplicates: 1; outbox pending: 0	транспортный дубль возможен, backlog закрывается	нужны идемпотентные потребители и outbox monitoring
Pub/Sub late subscriber	delivered=false	поздний подписчик не получает историю	Pub/Sub нельзя использовать как durable-журнал
WebSocket replay	received=3	reconnect восстанавливает события из stream	нужен last-seen marker и persisted stream
Cache stale	cached=100; actual=150; stale=true	cache-aside может вернуть устаревшее значение	кэш не является source of truth для экономики
Saga outcomes	completed=1; compensated=1; p95=22,370 мс	оба исхода наблюдаемы и проверяемы	Saga требует явных состояний и компенсаций

Содержательный результат fault-injection состоит в том, что каждый проверяемый отказ я связал с проектным решением. Повторная доставка callback требует Inbox; publish-no-ack требует идемпотентных потребителей; Pub/Sub loss требует persisted stream или восстановления из PostgreSQL; WebSocket reconnect требует last-seen marker; stale cache требует запрета на использование кэша как источника экономических решений; Saga требует явных состояний и компенсаций.

5.2 Evidence trace

Чтобы не оставить разрыв между экспериментом и рекомендациями, я сформировал evidence trace: каждому критерию сопоставил сценарий, метрику, артефакт, результат и вывод. Сводная версия приведена в таблице 5.2; полная JSON-форма сохранена в

results/latest/evidence_trace.json.

Таблица 5.2 – Трассировка доказательств от критерия к выводу

Критерий	Сценарий	Метрика	Артефакт	Вывод
Масштабируемость	серии 100/500/1000 для sync/outbox/chat	throughput и error rate	scenario runs, summary	горизонтально масштабируемые каналы нужны прежде всего для fan-out и worker-обработки событий
Надёжность	duplicate, publish-no-ack, Pub/Sub loss, replay, cache, Saga	duplicates, pending, delivered, stale, completed/compensated	summary, таблица отказов	надёжность достигается комбинацией Outbox, Inbox, Streams, Saga и source of truth
Производительность	latency и throughput серии	p50, p95, p99, throughput	scenario runs, таблица нагрузочных серий	overhead Outbox приемлем только при контроле backlog и retries
Сложность внедрения	реализованные компоненты и state machine	число компонентов, состояний и контрактов	comparison matrix CSV	сложные паттерны применяются только там, где локальная транзакция не закрывает риск
Стоимость эксплуатации	stateful-зависимости и retained history	stream length, attempts, worker, PostgreSQL, Redis	comparison matrix CSV, summary	стоимость определяется политикой хранения stream/outbox и наблюдаемостью worker
Наблюдаемость	/metrics/summary и артефакты	backlog age, duplicates, Saga duration, cache stale	summary, evidence trace JSON	рекомендация считается проверенной только при наличии наблюдаемой метрики

5.3 Наблюдаемость и проверяемые метрики

При проверке стенда я отдельно выделил наблюдаемость. Для архитектуры серверной платформы текстовой MMORPG это не вспомогательное свойство, а условие управляемости. Если система не измеряет возраст Outbox backlog, число повторных публикаций, stream lag, длительность Saga и устаревшие значения кэша, то эксплуатационная команда не сможет отличить корректную повторную доставку от потери события или зависшего worker. В стенде эта связь отражена через endpoint /metrics/summary; ключевые метрики приведены в таблице 5.3.

Таблица 5.3 – Метрики наблюдаемости, использованные в исследовании

Метрика	Значение в запуске	Что подтверждает
inbox_count	16003	команды и callback фиксируются для дедупликации
outbox_published	8004	worker реально публиковал исходящие события
outbox_attempts_total	8005	повторная попытка после publish-no-ack наблюдаема
outbox_pending	0	backlog закрыт после завершения эксперимента
outbox_oldest_pending_ms	0	незакрытых старых событий нет
redis_stream_len	8005	persisted stream содержит историю для replay
redis_duplicate_events	1	транспортный дубль воспроизведён и измерен
chat_messages	8000	чатовая нагрузка обработана полностью
chat_stream_len	8000	чатовые события сохранены в stream
saga_completed/ saga_compensated	1 / 1	оба исхода Saga достигнуты
saga_duration_p95_ms	22,370	длительность Saga наблюдаема

5.4 Проверка терминологии, источников и ограничений валидности

Терминологию я уточнял по уровням гарантий. At-least-once означает возможную повторную доставку сообщения. Идемпотентность означает отсутствие дополнительного побочного эффекта при повторной обработке. Однократный бизнес-результат достигается за счёт уникальных ключей, транзакционных ограничений, Inbox и дедупликации, а не за счёт абсолютной гарантии транспорта. Такая формулировка согласуется с паттерном Idempotent Receiver и описаниями delivery guarantees для Outbox/Inbox [11, 13, 22].

Источники я проверял по библиографическим сведениям, авторству, году публикации, URL или DOI, а также по применимости к конкретному утверждению. Дата обращения для электронных источников — 21.06.2026.

Ограничения валидности я зафиксировал отдельно. Во-первых, стенд проверяет архитектурные свойства, а не поведение полного production-кластера с геораспределением, реальной авторизацией, платежами и миллионами WebSocket-подключений. Во-вторых, локальные latency и throughput зависят от машины, Docker и фоновой нагрузки, поэтому используются как воспроизводимые значения конкретного стенда. Во-третьих, оценки сложности внедрения и стоимости эксплуатации опираются на реализованные компоненты стенда и источники; при переходе к промышленной системе они должны уточняться с учётом требований к SLA, резервированию, хранению истории и мониторингу.

5.5 Практические рекомендации

Практические рекомендации я включал в итоговый набор только при наличии проверяемого основания. Поэтому каждая рекомендация в таблице 5.4 связана с экспериментом, метрикой или источником; рекомендации без такого основания в итоговый набор не включались.

Таблица 5.4 – Практические рекомендации по проектированию платформы

№	Рекомендация	Основание	Доказательное значение
R1	PostgreSQL использовать как source of truth для экономики, инвентаря, прогресса и аудита	sync-серии, cache stale, документация PostgreSQL	кэш вернул 100 при фактическом значении 150; критичное решение должно идти в БД
R2	Каждую внешнюю команду и callback снабжать idempotency key и фиксировать в Inbox	duplicate callback, Idempotent Receiver	5 повторов дали один бизнес-результат и 4 зарегистрированных дубля
R3	Исходящие доменные события публиковать через Transactional Outbox и проектировать потребителей идемпотентными	publish-no-ack, Outbox/Inbox sources	транспортный дубль появился, но outbox_pending=0 после повторной обработки
R4	Redis Pub/Sub использовать только для transient fan-out, а Redis Streams — для replay и контролируемой доставки	Pub/Sub loss, WebSocket replay, Redis docs	поздний подписчик не получил сообщение; WebSocket replay получил 3 события
R5	Saga применять только для долгих экономических операций с явными компенсациями	Saga scenario, Microsoft/InfoQ	получены оба исхода: completed=1 и compensated=1
R6	В мониторинг включить backlog age, stream lag, duplicates, Saga duration, cache stale и error rate	/metrics/summary, evidence trace	без этих метрик невозможно доказательно связать отказ с архитектурной причиной

6 Выполнение этапа 5: оформление отчётных документов

Этап 5 я выполнял в течение всего периода НИР с 02.03.2026 по 21.06.2026. После подготовки экспериментального стенда и получения результатов я собрал письменный отчёт так, чтобы основная часть отражала именно выполнение этапов 2–4, а приложения содержали вспомогательные материалы. Ключевые интерпретированные результаты — сравнительную матрицу, графики нагрузочных серий, профиль критериев, таблицу метрик, evidence trace и рекомендации — я вынес в основную часть, поскольку на

них непосредственно ссылается текст и по ним проводится доказательное сравнение. В приложениях я оставил команды запуска, структуру стенда, перечень raw-артефактов и протокол проверки.

Оформление я выполнил в соответствии с методическим пособием ИТМО [4]: сохранил титульный лист, содержание, список сокращений, введение, основную часть, заключение, список источников и приложения. Отчёт собирается командой `./build_pdf.sh`; сборку я проверял по отсутствию ошибок LaTeX, неопределённых ссылок и неопределённых цитирований. Экспериментальные артефакты создаются командами `make experiment` и `make verify-results`; полные команды и структура стенда приведены в приложении 8. Для обсуждения с научным руководителем я подготовил итоговые материалы: текст отчёта, экспериментальные таблицы, графики, raw-артефакты и ссылку на опубликованный репозиторий стенда.

7 Заключение

В ходе научно-исследовательской работы я выполнил этапы 2–5 задания на практику. На этапе 2 я проанализировал предметную область серверных платформ текстовых MMORPG, определил объект, предмет, цель, задачи, классы сценариев, критерии анализа и источниковую базу. На этапе 3 я исследовал архитектурные решения и разработал практический стенд, на котором выполнил серии 100/500/1000 операций для sync/outbox/chat и fault-сценарии для duplicate callback, publish-no-ack, Pub/Sub loss, WebSocket replay, stale cache и Saga. На этапе 4 я верифицировал выводы через источники, терминологию, матрицу отказов, evidence trace и проверяемые метрики. На этапе 5 я подготовил письменный отчёт и приложения со вспомогательными материалами.

Цель работы достигнута: я обосновал архитектурную комбинацию PostgreSQL + Transactional Outbox/Inbox + Redis Streams/Pub/Sub + WebSocket gateway + Saga + cache-aside с разграничением областей применения. PostgreSQL используется как источник истины для критичного состояния; Outbox и Inbox обеспечивают восстановимую публикацию и защиту от повторного бизнес-эффекта; Redis Pub/Sub применяется только для transient fan-out, а Redis Streams — для replay; WebSocket обеспечивает

realtime-доставку при наличии механизма восстановления; Saga применяется к долгим экономическим цепочкам; cache-aside ограничивается данными, где допустима устарелость или есть проверка через source of truth.

Итоговые рекомендации я подтвердил экспериментальными артефактами: `summary.json`, `scenario_runs.csv`, `comparison_matrix.csv`, `evidence_trace.json`, таблицей нагрузочных серий, таблицей отказов и таблицей наблюдаемости. Команда `make verify-results` проверяет наличие артефактов, структуру серий, отсутствие незакрытого Outbox backlog и выполнение ключевых инвариантов. Отзыв руководителя и оценка за практику оформляются руководителем в модуле «Практика» в период 22.06.2026–27.06.2026; в отчёте они не указываются как установленный факт.

8 Список использованных источников

1. *Fette I., Melnikov A.* The WebSocket Protocol. — 2011. — DOI: 10.17487/RFC6455. — URL: <https://www.rfc-editor.org/info/rfc6455> (дата обр. 21.06.2026). RFC 6455.
2. *Горинов Д. А.* Архитектурные паттерны серверных платформ для текстовых многопользовательских онлайн-игр : Аналитический обзор / Университет ИТМО. — Санкт-Петербург, 2026. — Подготовлено в рамках научно-исследовательской работы предыдущего семестра.
3. *Горинов Д. А.* MMORPG Architecture Research Stand. — 2026. — URL: <https://github.com/gorinovdan/mmorpg-architecture-research> (дата обр. 21.06.2026) ; Исследовательский стенд для проверки Outbox/Inbox, Redis Streams/Pub/Sub, WebSocket reconnect, Saga и cache-aside.
4. Производственная практика магистрантов: организация и проведение / Т. А. Маркина [и др.]. — Санкт-Петербург : Университет ИТМО, 2020. — 50 с. — URL: <https://books.ifmo.ru/file/pdf/2622.pdf> (дата обр. 21.06.2026).
5. *Kasenides N., Paspallis N.* A Systematic Mapping Study of MMOG Backend Architectures // Information. — 2019. — Т. 10, № 9. — DOI: 10.3390/info10090264. — URL: <https://www.mdpi.com/2078-2489/10/9/264> (дата обр. 21.06.2026).
6. *Zhang K., Kemme B., Denault A.* Persistence in Massively Multiplayer Online Games // Proceedings of the 7th ACM SIGCOMM Workshop on Network and System Support for Games. — New York : ACM, 2008. — С. 53—58. — DOI: 10.1145/1517494.1517505. — URL: <https://www.cs.mcgill.ca/~adenau/pub/persistence.pdf> (дата обр. 21.06.2026).

7. *Zhang K., Kemme B.* Transaction Models for Massively Multiplayer Online Games // 2011 IEEE 30th International Symposium on Reliable Distributed Systems. — IEEE Computer Society, 2011. — C. 31—40. — DOI: 10.1109/SRDS.2011.13. — URL: <https://espace2.etsmtl.ca/id/eprint/16435/> (дата обр. 21.06.2026).
8. *Palant W., Griwodz C., Halvorsen P.* Consistency Requirements in Multiplayer Online Games // Proceedings of the 5th ACM SIGCOMM Workshop on Network and System Support for Games. — ACM, 2006. — C. 51. — DOI: 10.1145/1230040.1230061. — URL: <https://web-backend.simula.no/sites/default/files/publications/Palant.2006.2.pdf> (дата обр. 21.06.2026).
9. *Li F. W. B., Li L. W. F., Lau R. W. H.* Supporting Continuous Consistency in Multiplayer Online Games // Proceedings of the 12th Annual ACM International Conference on Multimedia. — ACM, 2004. — C. 388—391. — DOI: 10.1145/1027527.1027619. — URL: <https://frederickli.webspace.durham.ac.uk/wp-content/uploads/sites/103/2021/04/mm04-sync.pdf> (дата обр. 21.06.2026).
10. *Diao Z.* Consistency Models for Cloud-Based Online Games: The Storage System's Perspective // 25th GI-Workshop on Foundations of Databases. — Ilmenau, 2013. — URL: https://ceur-ws.org/Vol-1020/paper_03.pdf (дата обр. 21.06.2026).
11. *Kleppmann M.* Designing Data-Intensive Applications: The Big Ideas Behind Reliable, Scalable, and Maintainable Systems. — Sebastopol : O'Reilly Media, 2017. — ISBN 978-1449373320.
12. *Hohpe G., Woolf B.* Enterprise Integration Patterns: Designing, Building, and Deploying Messaging Solutions. — Boston : Addison-Wesley, 2004. — ISBN 978-0321200686.
13. *Hohpe G., Woolf B.* Idempotent Receiver / Enterprise Integration Patterns. — 2004. — URL: <https://www.enterpriseintegrationpatterns.com/>

- patterns/messaging/IdempotentReceiver.html (дата обр. 21.06.2026).
14. *Hohpe G., Wolf B. Correlation Identifier / Enterprise Integration Patterns.* — 2004. — URL:
<https://www.enterpriseintegrationpatterns.com/patterns/messaging/CorrelationIdentifier.html>
(дата обр. 21.06.2026).
 15. *Vogels W. Eventually Consistent // Communications of the ACM.* — 2009. — Т. 52, № 1. — С. 40—44. — DOI:
10.1145/1435417.1435432. — URL: <https://cacm.acm.org/practice/eventually-consistent/>
(дата обр. 21.06.2026).
 16. *Redis. Redis Streams / Redis Documentation.* — 2026. — URL: <https://redis.io/docs/latest/develop/data-types/streams/>
(дата обр. 21.06.2026).
 17. *Redis. Redis Pub/Sub / Redis Documentation.* — 2026. — URL:
<https://redis.io/docs/latest/develop/pubsub/>
(дата обр. 21.06.2026).
 18. *PostgreSQL Global Development Group. High Availability, Load Balancing, and Replication / PostgreSQL Documentation.* — 2026. — URL: <https://www.postgresql.org/docs/current/high-availability.html> (дата обр. 21.06.2026).
 19. *PostgreSQL Global Development Group. Table Partitioning / PostgreSQL Documentation.* — 2026. — URL:
<https://www.postgresql.org/docs/current/ddl-partitioning.html> (дата обр. 21.06.2026).
 20. *Microsoft Azure Architecture Center. Event-driven architecture style / Microsoft Learn.* — 07.03.2026. — URL:
<https://learn.microsoft.com/en-us/azure/architecture/guide/architecture-styles/event-driven> (дата обр. 21.06.2026).

21. *Richardson C.* Pattern: Transactional Outbox / Microservices.io. — 2026. — URL: <https://microservices.io/patterns/data/transactional-outbox.html> (дата обр. 21.06.2026).
22. *Dudycz O.* Outbox, Inbox patterns and delivery guarantees explained / Event-Driven.io. — 30.12.2020. — URL: https://event-driven.io/en/outbox_inbox_patterns_and_delivery_guarantees_explained/ (дата обр. 21.06.2026).
23. *Microsoft Azure Architecture Center.* Saga distributed transactions pattern / Microsoft Learn. — 2026. — URL: <https://learn.microsoft.com/en-us/azure/architecture/patterns/saga> (дата обр. 21.06.2026).
24. *Morling G.* Saga Orchestration for Microservices Using the Outbox Pattern / InfoQ. — 25.02.2021. — URL: <https://www.infoq.com/articles/saga-orchestration-outbox/> (дата обр. 21.06.2026).
25. *Abyl Realtime.* WebSocket architecture best practices: Designing scalable realtime systems / Abyl. — 05.11.2024. — URL: <https://abyl.com/topic/websocket-architecture-best-practices> (дата обр. 21.06.2026).
26. *O’Riordan M.* WebSockets at Scale: Architecture for Millions of Connections / WebSocket.org. — 02.09.2024. — URL: <https://websocket.org/guides/websockets-at-scale/> (дата обр. 21.06.2026) ; Обновлено 10.03.2026.
27. *Alpatov A. N., Iurov I. I.* Algorithm and Software Implementation of Real-Time Collaborative Editing of Graphical Schemes Using Socket.IO Library // Программные системы и вычислительные методы. — 2024. — № 1. — С. 10—19. — DOI: 10.7256/2454-0714.2024.1.70173. — URL: <https://journals.rcsi.science/2454-0714/article/view/359422> (дата обр. 21.06.2026).

28. *Fedulov A. A.* Designing the Architecture of the Client-Server Interaction Protocol for Web Applications Based on WebSocket // Программные системы и вычислительные методы. — 2025. — № 3. — С. 20—30. — DOI: 10.7256/2454-0714.2025.3.75392. — URL: <https://journals.rcsi.science/2454-0714/article/view/359337> (дата обр. 21.06.2026).
29. *Tessier J.* Why your caching strategies might be holding you back (and what to consider next) / Redis Developer Blog. — 13.06.2025. — URL: <https://redis.io/blog/why-your-caching-strategies-might-be-holding-you-back-and-what-to-consider-next/> (дата обр. 21.06.2026).
30. *Fowler M.* BlueGreenDeployment / martinfowler.com. — 01.03.2010. — URL: <https://martinfowler.com/bliki/BlueGreenDeployment.html> (дата обр. 21.06.2026).

Репозиторий, команды запуска и структура стенда

Публичный репозиторий исследовательского стенда:

github.com/gorinovdan/mmorpg-architecture-research

Команды запуска и проверки стенда

Команда	Назначение
<code>make deps</code>	Создать Python-окружение и установить зависимости для графиков.
<code>make test</code>	Запустить Go-тесты, проверить Docker Compose и Python-скрипты.
<code>make up</code>	Собрать и поднять PostgreSQL, Redis, backend и worker.
<code>make migrate</code>	Проверить применение миграций backend при старте.
<code>make experiment</code>	Выполнить серии нагрузки и fault-injection сценарии.
<code>make verify-results</code>	Проверить инварианты, матрицы результатов и evidence trace.
<code>make down</code>	Остановить контейнеры стенда.

Структура стенда

Путь	Назначение
<code>cmd/researchd</code>	Backend и worker: HTTP API, WebSocket, PostgreSQL, Redis, Outbox, Inbox, Saga, cache-aside.
<code>cmd/wscheck</code>	Проверка WebSocket replay после подключения.
<code>internal/expstats</code>	Проверяемые функции расчёта процентилей и дублей.
<code>scripts/run_experiments.py</code>	Запуск сценариев нагрузки и отказов, сбор CSV/JSON и вспомогательных графиков.
<code>scripts/verify_results.py</code>	Проверка инвариантов, артефактов и evidence trace.
<code>docs/architecture.md</code>	Описание границ стенда, компонентов, потоков данных и метрик.
<code>docs/experiments.md</code>	Методика экспериментов, артефакты и правила интерпретации.
<code>.github/workflows/ci.yml</code>	GitHub Actions для Go-тестов, Docker smoke test и проверки артефактов.

Raw-сводка экспериментальных артефактов

Приложение содержит сокращённую сводку исходных артефактов стенда. Интерпретация результатов приведена в основной части отчёта; здесь зафиксированы файлы, по которым проверяется воспроизводимость.

Файлы результатов `results/latest`

Файл	Содержание
<code>summary.json</code>	Сводка запуска от 21.06.2026: seed, серии нагрузки, fault-сценарии, финальные метрики, матрица сравнения, evidence trace и рекомендации.
<code>metrics.csv</code>	Нормализованная выгрузка метрик в формате <code>scenario, metric, value, unit</code> .
<code>scenario_runs.csv</code>	45 отдельных запусков: 3 сценария, 3 размера нагрузки, 5 повторов.
<code>comparison_matrix.csv</code>	Экспериментальная матрица решений по критериям.
<code>criteria_scores.csv</code>	Развёрнутая форма оценок по критериям для проверки итоговой матрицы.
<code>evidence_trace.json</code>	Машиночитаемая цепочка «критерий — сценарий — метрика — результат — вывод».
<code>recommendations.json</code>	Рекомендации с указанием экспериментального основания.

Сокращённая raw-сводка `summary.json`

Блок	Поле	Значение
Параметры запуска	seed; load sizes; repeats	4116; 100/500/1000; 5
Объём серий	scenario runs	45
Чат	messages; stream length	8000; 8000
Inbox/Outbox	inbox count; published; attempts; pending	16003; 8004; 8005; 0
Redis Streams	stream length; duplicate events	8005; 1
Pub/Sub	delivered to late subscriber	false
Cache-aside	cached; actual; stale	100; 150; true
Saga	completed; compensated; p95 duration	1; 1; 22,370 mc
WebSocket replay	received	3

Протокол проверки источников, сборки и оформления

Протокол проверки отчёта, источников и стенда

Проверка	Выполненное действие	Результат
Структура по методическому пособию	Проверены титульный лист, содержание, список сокращений, введение, основная часть, заключение, источники и приложения	Соответствует
Оформление	Проверены формат A4, поля 30/15/20/20 мм, Times New Roman 14 пт, полуторный интервал, ссылки в квадратных скобках и расположение приложений после источников	Соответствует
Источники	Проверены библиографические сведения, URL/DOI, применимость утверждений и дата обращения 21.06.2026	Подтверждено
Стенд	Выполнены <code>make deps</code> , <code>make test</code> , <code>make up</code> , <code>make migrate</code> , <code>make experiment</code> , <code>make verify-results</code>	Успешно
Детерминированность структуры	Эксперимент повторён с seed 4116; проверены структура артефактов и инварианты, абсолютные latency не фиксировались как неизменные	Подтверждено
Сборка PDF	Выполнен <code>./build_pdf.sh</code> ; проверены <code>report.log</code> и <code>report.blg</code> на ошибки, неопределённые ссылки и цитирования	Успешно
Визуальная проверка	Проверены титульный лист, содержание, план-график, методика, таблицы, рисунки, заключение и начало приложений	Выполнено
Текстовые ограничения	Проверено отсутствие служебных маркеров, упоминаний предыдущих внутренних документов и некорректных утверждений об однократной транспортной доставке	Выполнено
GitHub	Проверена доступность репозитория, публикация обновлённых артефактов и последний запуск GitHub Actions	Подтверждено